INDIAN INSTITUTE OF INFORMATION TECHNOLOGY, MANIPUR
CLOUD COMPUTING | CS3081 | MINI PROJECT

Cloud Command Parser

Knowledge distillation and deployment of a lightweight command-parsing language model on a local cloud

STUDENT	ROLL NUMBER	PROGRAMME / SECTION	SEMESTER
Rishabh Bhardwaj	230101059	B.Tech CSE Core / A	VII
INSTRUCTOR	**DESIGNATION**	**DEPARTMENT**	**INSTITUTE**
Dr Salam Michael Singh	Assistant Professor	CSE	IIIT Manipur

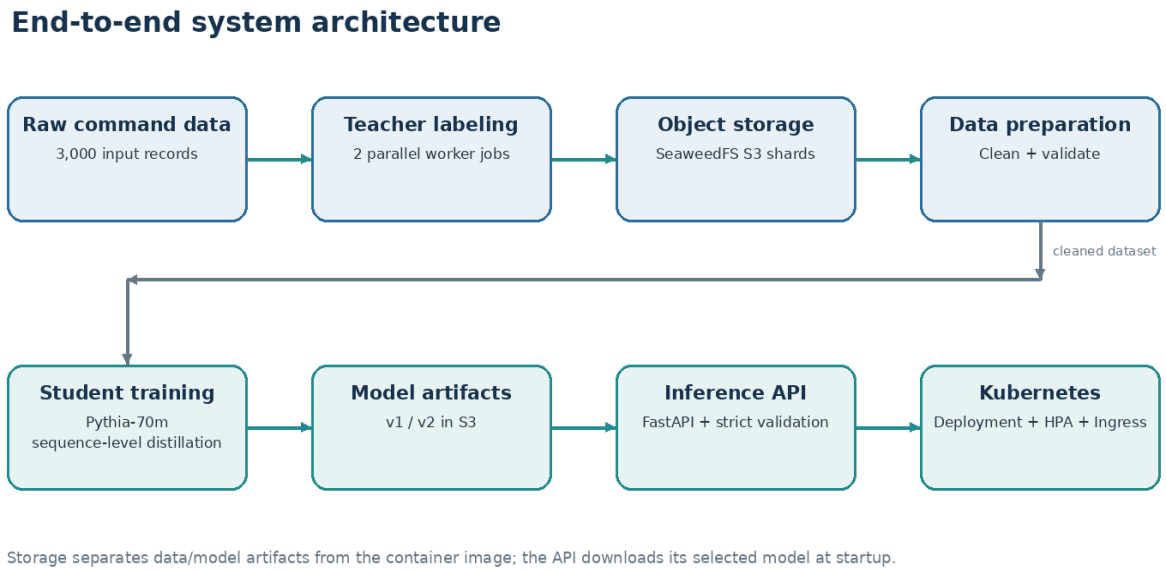
Instructor profile: Postdoctoral work at IIT Delhi; PhD, NIT Silchar. Areas include deep learning, NLP, machine translation, multilingual NLP and generative AI.

Executive summary

This project builds a small command parser for eight intents and three optional slots. A Phi-3.5 teacher labels a 3,000-command corpus; two parallel workers store raw responses in SeaweedFS object storage. After validation and deduplication, the first 1,500 cleaned examples fine-tune EleutherAI/Pythia-70M on CPU. The resulting model is served by FastAPI in a container and deployed to a local k3d Kubernetes cluster, where model artefacts are downloaded from S3-compatible storage at startup.

2,828	70M	50/50	11 s
cleaned commands	student parameters	sequential API requests	median pod cold start

End-to-end architecture



Storage separates data/model artifacts from the container image; the API downloads its selected model at startup.

Model weights are stored separately from the service image. At startup, each inference pod downloads its selected model and tokenizer from SeaweedFS.

Task 1 | Object storage and parallel teacher labelling

Implementation

A k3d cluster is defined in **cluster/k3d.yaml**. SeaweedFS runs in namespace **data** and exposes an S3-compatible endpoint. The **datasets** and **models** buckets separate source/label data from trained artefacts. The original 3,000-row command file is uploaded to the datasets bucket.

Teacher and sharding

Two worker Jobs run concurrently in namespace **pipeline**, with **SHARD_COUNT=2** and distinct indices 0 and 1. A worker selects zero-based input lines where **line_number % 2 == SHARD_INDEX**. Each worker writes its own JSONL object containing the original command and the teacher response exactly as returned by Ollama. The teacher is Phi-3.5 with temperature 0 and seed 42; the prompt is stored at **prompts/teacher_prompt.txt**.

Cleaning outcome	Count
Input commands	3,000
Invalid JSON replies	0
Replies with incorrect keys	0
Invalid intent	0
Invalid slot values	126
Duplicate commands removed	46
Records retained	2,828

Accounting check: 3,000 = 0 + 0 + 0 + 126 + 46 + 2,828. The cleaner checks the exact four-key schema, allowed intents, slot text against the original command, and duplicate commands (keeping the smallest ID).

Storage outputs and reproducibility

- Raw worker shards are stored as **datasets/teacher/shard-0.jsonl** and **datasets/teacher/shard-1.jsonl**.
- The cleaned, ID-ordered dataset is stored at **datasets/v1/train.jsonl** with SHA-256 recorded in **datasets/v1/train.sha256**.
- The shard validator checks that every source ID appears exactly once, no IDs repeat, and each shard has the expected 1,500 rows.

The 2,828 retained records represent the teacher-labelled training corpus. The validation set is kept separate and is not used by the cleaner or student training process.

Task 2 | Training the student by sequence-level distillation

Training setup

EleutherAI/Pythia-70M is fine-tuned on CPU using PyTorch and Hugging Face Transformers. Each input is formatted as **Command: <command>** followed by **JSON: <teacher response>**. The training script reads its parameters from **training/config.yaml** and uses the first 1,500 records of the cleaned dataset. Seed 42, learning rate 5×10^{-5}, batch size 2, maximum sequence length 64 and three epochs are configured for v1; v2 adds a fourth epoch.

Parameter	Configured value
Base model	EleutherAI/pythia-70m
Device / seed	CPU / 42
Learning rate / batch	0.00005 / 2
Sequence length	64 tokens
Epochs	v1: 3; v2: 4
Runtime versions	Python 3.14.4; torch 2.15.0+cpu; transformers 5.22.2; PyYAML 6.0.3

Validation results (100 labelled examples)

Model	JSON validity	Intent	Date	Time	Place	Full frame
Teacher	100%	83%	85%	88%	91%	56%
Untrained Pythia	0%	0%	0%	0%	0%	0%
Student v1	96%	71%	80%	88%	89%	51%
Student v2	98%	78%	83%	89%	80%	48%

Training loss and interpretation

Average epoch loss fell from 0.386887 to 0.201063 for v1; the fourth v2 epoch reached 0.146823. On the separate validation set, both students improved substantially over untrained Pythia. V1 reached 51% full-frame accuracy, above half of the teacher's 56% score (28%). V2 improved JSON validity and intent accuracy but its full-frame score was 48%, so additional training did not improve every metric.

> Distillation distinction: sequence-level distillation trains the student on complete teacher-generated output sequences. Logit-based distillation matches the teacher's token-level probability distributions. The Ollama teacher interface used here exposes generated text, not teacher logits, so sequence-level targets are the practical method. A limitation is that student quality inherits teacher-label noise and is evaluated on a small 100-example validation set.

Both trained model/tokenizer versions are stored in object storage and published as release archives. The v1 archive checksum is recorded in **models/v1/model.sha256**; model weights are excluded from Git.

Task 3 | Containerised inference service

Service contract and model loading

The FastAPI application listens on port 8000. It provides **POST /parse**, **GET /healthz** and **GET /readyz**. A successful parse returns only the four-key JSON object and sets **X-Model-Version** and **X-Served-By**. Readiness remains HTTP 503 until the tokenizer and model have loaded. At startup the service downloads the selected model and tokenizer from the S3-compatible endpoint using credentials mounted from a Kubernetes Secret.

Endpoint	Behaviour
POST /parse	Returns schema-valid {intent,date,time,place} JSON; includes model and pod headers. Invalid output returns HTTP 500 with {"error":"invalid_model_output"}.
GET /healthz	Liveness endpoint; HTTP 200 while process is alive.
GET /readyz	Readiness endpoint; HTTP 503 before model load and HTTP 200 once ready.

Configuration, image and safety checks

- Application settings are supplied through MODEL_URI, MODEL_VERSION, S3_ENDPOINT and MAX_NEW_TOKENS; FAULT_INJECT=1 forces an invalid string to verify strict response validation.
- AWS credentials are sourced from Kubernetes Secrets; they are not embedded in the image or committed to the repository.
- Dependency versions are pinned. The CPU-only PyTorch wheel avoids GPU libraries. The image contains application code and schema, not model weights.
- The measured image size is 1,836,718,325 bytes (about 1.71 GiB), below the 2 GB assignment limit.

Sequential container benchmark under 1 vCPU / 1 GiB

Measure	Observed result
Requests	50 total; 50 HTTP 200; 0 failed; 50 schema-valid
Duration / throughput	19.011 s / 2.63 requests per second
Latency	Average 380.10 ms; p50 369.28 ms; p95 426.19 ms; max 587.14 ms
Peak observed memory	529 MiB (kubectl top sampling)
Model / pod	v1; one pod served all 50 sequential requests

Fault-injection check: with FAULT_INJECT=1, /parse returned HTTP 500 and the exact body {"error":"invalid_model_output"}. The service validates every model output before returning success.

Task 4 | Kubernetes deployment, scaling and reliability

Deployment design

The inference service is deployed in namespace **app** as a two-replica Deployment behind a ClusterIP Service and Traefik Ingress (host **parser.local**). Each pod requests 500m CPU and 1 GiB memory and is limited to 1 CPU and 1 GiB. Startup, liveness and readiness probes are configured. The HPA has a minimum of one replica, maximum of two and 70% average CPU target.

Test	Measured result	Assessment against handout
Initial deployment / probes	2 replicas; readiness waits for model load	Implemented; readiness gates traffic
HPA	Scaled 1 → 2 under load, then back to 1	Target 70%; observed scale up/down
Rolling update v1 → v2	217/220 successful; 3 HTTP 502 (1.36%)	Above 0.5% failure target
Rollback v2 → v1	142/145 successful; 3 HTTP 502 (2.07%); final model v1	Version rollback verified; transient failures observed
Pod deletion recovery	150/150 successful; replacement pod became Ready	Meets 2% maximum failure target
Cold start (3 runs)	11 s, 11 s, 11 s; median 11 s; S3 download confirmed each run	Three measurements recorded
Load balancing	Two ready pods served 25/50 (50%) each	Both exceed 30% minimum

Reliability interpretation

The pod-deletion experiment restored the Deployment without losing any request in the 150-request window. Cold-start logs show each replacement pod downloading six model files from object storage before becoming ready. The load-balancing run distributed successful requests evenly. The rolling update and rollback each had transient HTTP 502 responses while endpoints changed; the measured failure rates are shown explicitly rather than described as zero-downtime.

> Local-cluster limitation: k3d ran on one laptop as a single-node cluster. These tests demonstrate pod rescheduling and replica management, not node-level high availability or fault tolerance across multiple machines.

A production-oriented next step is to test connection draining (for example, a preStop delay and graceful shutdown), then repeat rolling updates under the instructor’s load test and verify the ≤0.5% failure threshold.

Task 5 | Subnet allocation and stateless NACL

VLSM allocation

The parent block is 10.20.0.0/16. Subnets are allocated from the largest host requirement to the smallest, beginning at 10.20.0.0 and placed sequentially with the smallest suitable prefix.

Subnet	Hosts needed	Network / prefix	Usable range	Broadcast	Usable
App	1,000	10.20.0.0/22	10.20.0.1 - 10.20.3.254	10.20.3.255	1,022
Public	250	10.20.4.0/24	10.20.4.1 - 10.20.4.254	10.20.4.255	254
Data	100	10.20.5.0/25	10.20.5.1 - 10.20.5.126	10.20.5.127	126
Management	20	10.20.5.128/27	10.20.5.129 - 10.20.5.158	10.20.5.159	30

The four subnets are non-overlapping, remain inside the /16 parent, and meet all host requirements. Total allocated address space is 1,440 addresses. The validation script checks alignment, overlap, containment, sequential allocation and usable-host capacity.

Public-subnet NACL

The NACL follows the handout exactly. Inbound rule 100 denies all protocols and ports from 198.51.100.0/24 before the web rules. Rules 110 and 120 allow TCP 80 and 443 from anywhere; rule 130 permits TCP 22 only from management; rule 140 permits TCP ephemeral ports 1024-65535 from the app subnet for return traffic. Outbound rules allow TCP 8000 to the app subnet and TCP 1024-65535 to any client for replies. Both directions end with an explicit default deny.

Direction	Rule	Match	Action
Inbound	100	Any protocol/port from 198.51.100.0/24	Deny
Inbound	110-120	TCP 80, 443 from anywhere	Allow
Inbound	130	TCP 22 from 10.20.5.128/27	Allow
Inbound	140	TCP 1024-65535 from 10.20.0.0/22	Allow
Outbound	100	TCP 8000 to 10.20.0.0/22	Allow
Outbound	110	TCP 1024-65535 to anywhere	Allow

> Simulation results: supplied starter packets 6/6; reply examples 2/2; additional policy-focused cases 18/18. The unit tests also load the actual NACL file and check the required allow/deny behaviour.

A NACL is stateless, so it does not remember an allowed connection. Reply packets need their own rules. Rule order matters because the first match wins: the abusive CIDR deny must be evaluated before the HTTP/HTTPS allow rules.

Task 6 | Reproducibility, testing and CI/CD

Bootstrap and repository structure

The repository contains the k3d configuration, object-storage values, teacher-worker manifests, training and prediction scripts, service Dockerfile, Kubernetes manifests, network rules, unit tests, evidence and AI_USE.md. The Makefile exposes **make up**, **make down**, **make train**, **make test** and **make lint**. The README gives the three-command clone/deploy sequence and warns that teardown deletes the entire k3d cluster and its object store.

> Reproducibility qualification: make up was run repeatedly against the already existing cluster and completed Helm upgrade, model archive checksum verification, S3 upload, image build/import and Deployment rollout. A completely fresh-cluster run was not performed, because deleting the populated cluster would destroy retained teacher shards and object-store evidence. The fresh-cluster creation path is present in scripts/bootstrap.sh but remains an unverified environment path.

Automated workflow

Stage	What is checked	Evidence
Lint	Ruff on service, tests and project tools	CI run green
Unit tests	Automated Python tests, including schema, scoring and NACL policy tests	18 tests pass locally after policy addition
Model evaluation	Downloads v1 model from GitHub Release and scores validation set	JSON validity 96% in earlier recorded v1 report
Quality gate	Requires ≥95% JSON validity; separate manual workflow tests untrained Pythia	Positive gate passed; negative-run URL pending dispatch
Secret scanning	Gitleaks scans repository history	No findings in recorded scan

The latest previously verified main CI run was **42519747672**, which passed Lint, Unit Tests, Secret Scanning and Model Evaluation/Quality Gates. The updated workflow now sequences the core jobs as lint → unit tests → model evaluation/quality gate. A separate manual workflow runs untrained Pythia-70M through the same 95% threshold to create the handout’s intentionally failing demonstration; its run URL will be added after dispatch.

Release assets and AI-use disclosure

The v1 and v2 model/tokenizer archives and checksum file are attached to GitHub Release v1.0.0; model weights are not committed to Git. The repository includes AI_USE.md, which describes AI assistance and the checks performed on generated code and advice. The teardown command is implemented, but it was not executed against the evidence-bearing local cluster.

Task 7 | Performance, cost and review

Two-minute Locust run: 4 users, spawn rate 4/s, zero wait, via Traefik Ingress

Metric	Measured value
Requests / successful / failed	194 / 186 / 8
Success / error rate	95.88% / 4.12%
Latency average / p50 / p95 / max	2441 / 2800 / 4800 / 10492 ms
Throughput	1.63 requests/s (all attempts); 1.54 successes/s
Replica A peak CPU / memory	1002m / 470 MiB
Replica B peak CPU / memory	873m / 582 MiB

The eight failures were HTTP 500 responses caused by model output failing JSON/schema validation; Locust recorded no client-side exceptions. The console summary and CSV differed by one request (195 versus 194), and the per-replica header counter recorded 187 successes versus 186 in the aggregate CSV. The aggregate CSV is used consistently for rates and cost calculations. CPU reached the one-core limit on one sample; sampled memory stayed below 1 GiB.

Cost estimate (hypothetical rate card)

Hourly replica cost = (1 vCPU × ₹3.50) + (1 GiB × ₹0.45) = ₹3.95/hour. Two replicas cost ₹7.90/hour. Successful requests per hour = 186 ÷ 120.5 × 3,600 = 5,556.85. Cost per 1,000 successful requests = (₹7.90 ÷ 5,556.85) × 1,000 = ₹1.42. This estimate excludes SeaweedFS, Traefik, control-plane, storage and networking.

SLO and error budget

For the 194-request measurement window, the selected availability SLO is ≥95% successful schema-valid HTTP 200 responses. The corresponding error budget is 5% × 194 = 9.7 request-equivalents. Eight failures consumed 8 ÷ 9.7 = 82.47% of that budget; 1.7 request-equivalents remained. Observed success was 95.88%, so this selected SLO was met. The latency target was p95 <5,000 ms; 4,800 ms met it. These are short lab measurements, not a production 30-day SLO.

> Rubric distinction: the separate assignment performance threshold is an error rate of at most 1%. The observed load-test error rate was 4.12%, so that threshold was not met in this run even though the 95% availability SLO was met. The p95 latency threshold of 5,000 ms was met.

Review: measured evidence, limitations and improvements

Concern	Evidence	Limitation / improvement
Data & storage	2,828/3,000 retained; two S3 shards	126 slot rejects, 46 duplicates; audit teacher labels and expand coverage.
Model quality	Teacher full-frame 56%; v1 51%, v2 48%; v2 JSON 98%	Student below teacher; improve label quality and compare more checkpoints.
Latency / throughput	p50 2.8 s; p95 4.8 s; 1.63 req/s	p95 near 5 s; optimize decoding and CPU generation.
Resources	Peak 1002m CPU; memory 582 MiB	CPU near limit; profile and tune HPA/requests.
Reliability / recovery	150/150 after pod deletion; 11 s cold start	3/220 rollout 502s (1.36%); add draining and retest; one-node cluster.
Security / cost	Gitleaks clean; ₹1.42/1,000 success	Cost omits supporting services; scope credentials and model full infra cost.

Conclusion. The project demonstrates the complete path from teacher-generated labels to a containerised, object-storage-backed service with Kubernetes scaling, recovery, network policy and CI checks. The evidence also identifies two clear next steps: reduce invalid model outputs under load and improve graceful endpoint draining during rollouts.

References

The following project handout resources and official documentation informed the implementation. Dataset attribution is retained in the repository README.

• [1] Kim, Y. and Rush, A. M. (2016). Sequence-Level Knowledge Distillation. EMNLP. https://aclanthology.org/D16-1139/

• [2] EleutherAI. Pythia-70M model card. https://huggingface.co/EleutherAI/pythia-70m

• [3] Microsoft / Ollama. Phi-3.5 Mini Instruct. https://ollama.com/library/phi3.5

• [4] Bastianelli et al. (2020). SLURP: A Spoken Language Understanding Resource Package. EMNLP. https://aclanthology.org/2020.emnlp-main.588/

• [5] k3d documentation. https://k3d.io/stable/

• [6] SeaweedFS project documentation. https://github.com/seaweedfs/seaweedfs

• [7] Kubernetes documentation: Deployments, Services, probes, Secrets and Horizontal Pod Autoscaling. https://kubernetes.io/docs/

• [8] Docker documentation: Build best practices and resource constraints. https://docs.docker.com/

• [9] FastAPI documentation: Docker deployment. https://fastapi.tiangolo.com/deployment/docker/

• [10] Gitleaks. https://github.com/gitleaks/gitleaks

• [11] SLURP dataset source and CC BY 4.0 licence information. https://github.com/pswietojanski/slurp

Repository: https://github.com/Rishabh-Bhardwaj19/cloud-command-parser
Release v1.0.0: https://github.com/Rishabh-Bhardwaj19/cloud-command-parser/releases/tag/v1.0.0
Latest previously verified CI run:
https://github.com/Rishabh-Bhardwaj19/cloud-command-parser/actions/runs/42519747672